

Desafío 1

Ratmovil

Integrantes: Pablo Varetto D.
Joaquin Poblete
Felicie Nguyen
Enrique Rebolledo C.
Marina Olmedo G.
Profesor: Harold Valenzuela Coloma
Auxiliar: Fernando Navarrete U.
Ayudantes: Valentina Abarca G
Fernanda Echeverría Rojas
Fecha de realización: 09 de Octubre de 2025
Fecha de entrega: 09 de Octubre de 2025
Santiago de Chile

Índice de Contenidos

1. Diagrama esquemático	1
2. Funciones, control y circuito	2
3. Propuestas de mejora	3
4. Código comentado	4

Índice de Figuras

1.1. Diagrama Esquemático Ratmovil	1
--	---

Índice de Códigos

1. Código principal del control del Ratmóvil	4
--	---

1. Diagrama esquemático

Se presenta el dibujo de la arquitectura.

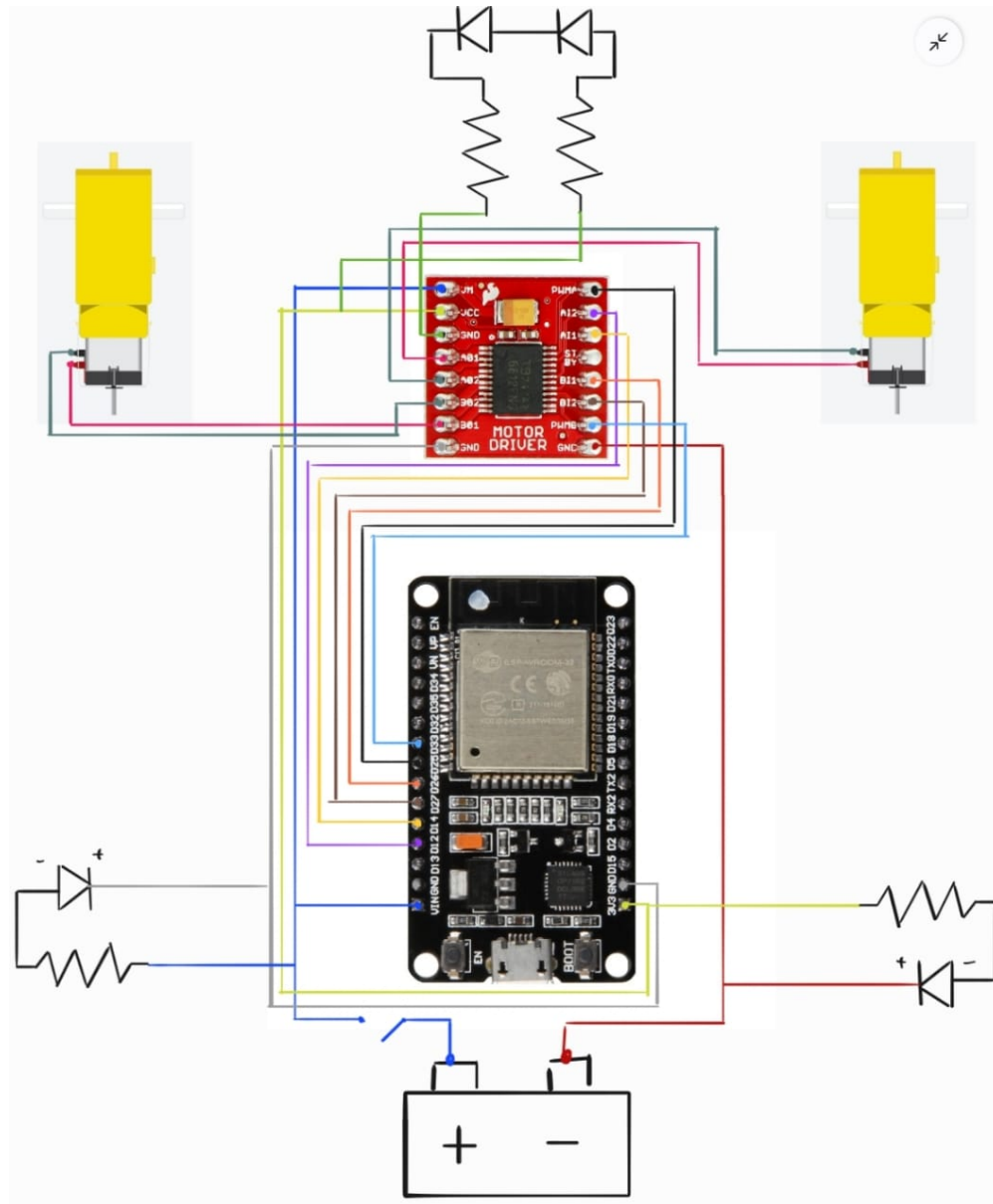


Figura 1.1: Diagrama Esquemático Ratmovil

2. Funciones, control y circuito

Una vez terminado el código, el Ratmóvil se pudo controlar a distancia utilizando la IP ya sea en un notebook o celular. El control del auto viene dado por las letras WASD y las flechas del teclado, las que cumplen las siguientes funciones:

- W: Avanzar.
- A: Girar hacia la izquierda.
- D: Girar hacia la derecha.
- S: Retroceder.

Adicionalmente, la página de control contiene un slider con la velocidad del auto, esta va desde 0 hasta poner el número 255, el equipo consideró que 180 era una velocidad rápida para probar el movimiento del Ratmóvil en el piso, 100 era una velocidad intermedia, y entre 40 y 60 era una velocidad baja, cabe destacar que bajando la velocidad a valores inferiores de 40 a veces el auto no llegaba a moverse.

Con el Ratmóvil listo y con control remoto por uno de los notebooks, se realizó un pequeño circuito en la segunda fecha del desafío con conos delimitando la pista, que no tenía gran tamaño pero sí hartas curvas. Con el slider de velocidad entre 40 y 60, el auto posee una velocidad adecuada para maniobrar y girar en el circuito sin salirse de este ni pasar a llevar los conos, por lo que se vio capaz de atravesar el circuito.

Posteriormente, los distintos equipos pegaron globos en la parte trasera de cada auto y monda-dientes en la delantera para realizar una batalla de globos, donde el objetivo era pinchar el globo del auto rival. En esta competencia, nuestro auto participó exitosamente en un inicio, utilizando velocidades superiores con el slider entre 100 y 180, aunque a partir de cierta distancia con el computador, las desconexiones aumentaron y dejó de ser viable el movimiento a control remoto.

3. Propuestas de mejora

- Se necesitaron dos fechas para terminar el código completamente, se necesitó tiempo para entender la naturaleza de los desafíos, se espera que para el siguiente ya habrá una mejor respuesta a las instancias, esto ya se experimentó pues en la segunda fecha todo fue más expedito.
- Una dificultad que perduró hasta el final fue la conexión de internet con el auto, al completar el código y cambiar las pilas seguían habiendo desconexiones, por lo que el problema se le atribuye al WiFi. Adicionalmente siempre existió un tiempo de retraso entre que se utilizaba el control remoto y que el auto efectivamente se moviese. No existe una solución clara a este problema, parecen ser desventajas inherentes a los componentes utilizados.
- En el equipo todos aportaron con el código inicialmente, posteriormente dos personas se centraron en él, mientras otro integrante se dedicó a anotar la forma del circuito, hubo instancias donde muchos se quedaron esperando las actualizaciones finales del código, pues tampoco es posible que más de una persona aporte a la vez a esa tarea. Sería de ayuda establecer más tempranamente los roles de cada uno en el desafío una vez es entregado el enunciado.
- Una dificultad que sucedió fue el cambio de autos debido a que uno no funcionaba correctamente, el auto de reemplazo funcionó, pero tenía una orientación diferente, por lo que se tuvieron que invertir las direcciones del movimiento de las ruedas y las luces LED en el código.
- Gracias al desafío 1, el equipo aprendió a implementar el código en el microcontrolador ESP 32, y utilizar el control remoto gracias a la IP.

4. Código comentado

Código 1: Código principal del control del Ratmóvil

```

1 #include <WiFi.h>
2 #include <WebServer.h>
3
4 // ===== Pines =====
5
6 // Motor A (lado izquierdo)
7 const int AIN1 = 23;
8 const int AIN2 = 4;
9 const int PWMA = 22;
10
11 // Motor B (lado derecho)
12 const int BIN1 = 18;
13 const int BIN2 = 19;
14 const int PWMB = 21;
15
16 // STBY (habilita el TB6612FNG)
17 const int STBY = 5;
18
19 // LEDs
20 const int LED_1 = 32;
21 const int LED_2 = 26;
22 const int LED_3 = 33;
23 const int LED_4 = 25;
24
25 // ===== PWM =====
26
27 const int pwmFree = 1000;
28 const int pwmResolution = 8;
29 const int pwmChannelA = 0;
30 const int pwmChannelB = 1;
31
32 // Velocidad base (0-255)
33 int motorSpeed = 180;
34
35 // ===== WiFi =====
36
37 // Insertar el nombre de su red WiFi
38 const char* ssid = "A15";
39 const char* password = "qilegueialuz";
40
41 // Servidor web en el puerto 80
42 WebServer server(80);
43
44 // ===== HTML (WASD + Flechas) =====
45 ↵ =====

```

```

46 const char* html = R"rawliteral(
47 <!DOCTYPE html>
48 <html>
49 <head>
50 <meta name="viewport" content="width=device-width, initial-scale=1">
51 <meta charset="UTF-8">
52 <style>
53 body { font-family: system-ui, Segoe UI, Roboto, Arial; margin: 16px; }
54 button { padding: 16px 18px; font-size: 18px; margin: 8px; border-radius: 10px; }
55 .grid { display: grid; gap: 8px; grid-template-columns: repeat(3, 120px); justify-content: center;
    ↪ margin-top: 16px; }
56 .kbd { padding: 2px 6px; border: 1px solid #ddd; border-radius: 6px; background: #ffffff; }
57 </style>
58 </head>
59 <body>
60 <h2>Desafío 1: "Fats & Rats"</h2>
61 <h3>Mecatrónica ME4250</h3>
62 <p>Usa <span class="kbd">W</span> <span class="kbd">A</span> <span class="kbd">S</span> <
    ↪ span class="kbd">D</span> o las flechas. <span class="kbd">Space</span> = Stop</p>
63
64 <div class="grid">
65 <div></div>
66 <button onclick="send('forward')"> Adelante</button>
67 <div></div>
68
69 <button onclick="send('left')"> Izquierda</button>
70 <button onclick="send('stop')"> Stop</button>
71 <button onclick="send('right')"> Derecha</button>
72
73 <div></div>
74 <button onclick="send('backward')"> Atrás</button>
75 <div></div>
76 </div>
77
78 <p>Velocidad: <span id="v">180</span>
79 <input id="spd" type="range" min="0" max="255" value="180" oninput="chgSpeed(this.value)">
80 </p>
81
82 <hr> <div class="grid" style="grid-template-columns: repeat(2, 1fr);">
83 <button onclick="send('function1')"> Función 1</button>
84 <button onclick="send('function2')"> Función 2</button>
85 </div>
86
87 <script>
88 const v = document.getElementById('v');
89 function send(cmd){ fetch('/') + cmd).catch(()=>{}); }
90 function chgSpeed(val){ v.textContent = val; fetch('/speed?val=' + val).catch(()=>{}); }
91
92 // Teclado: WASD y flechas
93 const mapDown = {
94   'w': 'forward', 'W': 'forward', 'ArrowUp': 'forward',

```

```

95     's': 'backward', 'S': 'backward', 'ArrowDown': 'backward',
96     'a': 'left', 'A': 'left', 'ArrowLeft': 'left',
97     'd': 'right', 'D': 'right', 'ArrowRight': 'right',
98     ' ': 'stop'
99 };
100
101 const pressed = new Set();
102 window.addEventListener('keydown', (e) => {
103     if (!(e.key in mapDown)) return;
104     if (!pressed.has(e.key)) {
105         pressed.add(e.key);
106         send(mapDown[e.key]);
107     }
108     e.preventDefault();
109 }, { passive: false });
110 window.addEventListener('keyup', (e) => {
111     if (!(e.key in mapDown)) return;
112     pressed.delete(e.key);
113     if (e.key !== ' ') { send('stop'); }
114     e.preventDefault();
115 }, { passive: false });
116 </script>
117 </body>
118 </html>
119 )rawliteral";
120
121 // ===== Util LEDs
122     ↳ =====
123
124 // apagar los LEDs
125 void ledsOff() {
126     digitalWrite(LED_1, LOW);
127     digitalWrite(LED_2, LOW);
128     digitalWrite(LED_3, LOW);
129     digitalWrite(LED_4, LOW);
130 }
131
132 // encender los LEDs
133 void ledsOn() {
134     digitalWrite(LED_1, HIGH);
135     digitalWrite(LED_2, HIGH);
136     digitalWrite(LED_3, HIGH);
137     digitalWrite(LED_4, HIGH);
138 }
139
140 void blinkLEDs(int times, int delayTime) {
141     for (int i = 0; i < times; i++) {
142         ledsOn();
143         delay(delayTime);
144         ledsOff();
145         delay(delayTime);

```



```
145     }
146 }
147
148 // LEDs del frente
149 void ledadelante() {
150     ledsOff();
151     // Allumer les LEDs avant
152     digitalWrite(LED_3, HIGH);
153     digitalWrite(LED_4, HIGH);
154 }
155
156 // LEDs de atras
157 void ledsatras() {
158     ledsOff();
159     // Allumer les LEDs arrière
160     digitalWrite(LED_1, HIGH);
161     digitalWrite(LED_2, HIGH);
162 }
163
164 // LEDs de la derecha
165 void ledDerecha() {
166     ledsOff();
167     // Allumer la LED droite
168     digitalWrite(LED_4, HIGH);
169 }
170
171 // LEDs de la izquierda
172 void ledizquierda() {
173     ledsOff();
174     // Allumer la LED gauche
175     digitalWrite(LED_3, HIGH);
176 }
177
178 // ===== Motores =====
179 void motorsStop() {
180     // Detiene la dirección
181     digitalWrite(AIN1, LOW);
182     digitalWrite(AIN2, LOW);
183     digitalWrite(BIN1, LOW);
184     digitalWrite(BIN2, LOW);
185     // Detiene la velocidad (opcional, pero buena práctica)
186     ledcWrite(PWMA, 0);
187     ledcWrite(PWMB, 0);
188 }
189
190 void motorsForward() {
191     // Motor A (Izquierdo) Adelante: AIN1 = HIGH, AIN2 = LOW
192     digitalWrite(AIN1, HIGH);
193     digitalWrite(AIN2, LOW);
194
195     // Motor B (Derecho) Adelante: BIN1 = HIGH, BIN2 = LOW
```

```
196     digitalWrite(BIN1, HIGH);
197     digitalWrite(BIN2, LOW);
198
199     // Aplica la velocidad configurada
200     ledcWrite(PWMA, motorSpeed);
201     ledcWrite(PWMB, motorSpeed);
202
203     // encender los LEDs correspondientes
204     ledadelante();
205 }
206
207 void motorsBackward() {
208     // Motor A (Izquierdo) Atrás: AIN1 = LOW, AIN2 = HIGH
209     digitalWrite(AIN1, LOW);
210     digitalWrite(AIN2, HIGH);
211
212     // Motor B (Derecho) Atrás: BIN1 = LOW, BIN2 = HIGH
213     digitalWrite(BIN1, LOW);
214     digitalWrite(BIN2, HIGH);
215
216     // Aplica la velocidad configurada
217     ledcWrite(PWMA, motorSpeed);
218     ledcWrite(PWMB, motorSpeed);
219
220     // encender los LEDs correspondientes
221     ledsatras();
222 }
223
224 void motorsLeft() { // Gira sobre eje: Motor A (Izquierdo) Atrás, Motor B (Derecho) Adelante
225     // Motor A (Izquierdo) Gira Atrás
226     digitalWrite(AIN1, LOW);
227     digitalWrite(AIN2, HIGH);
228
229     // Motor B (Derecho) Gira Adelante
230     digitalWrite(BIN1, HIGH);
231     digitalWrite(BIN2, LOW);
232
233     // Aplica la velocidad
234     ledcWrite(PWMA, motorSpeed);
235     ledcWrite(PWMB, motorSpeed);
236
237     // encender los LEDs correspondientes
238     ledizquierda();
239 }
240
241 void motorsRight() { // Gira sobre eje: Motor A (Izquierdo) Adelante, Motor B (Derecho) Atrás
242     // Motor A (Izquierdo) Gira Adelante
243     digitalWrite(AIN1, HIGH);
244     digitalWrite(AIN2, LOW);
245
246     // Motor B (Derecho) Gira Atrás
```

```
247     digitalWrite(BIN1, LOW);
248     digitalWrite(BIN2, HIGH);
249
250     // Aplica la velocidad
251     ledcWrite(PWMA, motorSpeed);
252     ledcWrite(PWMB, motorSpeed);
253
254     // encoder los LEDs correspondientes
255     ledDerecha();
256 }
257
258 // ===== Funciones extras
259     ↪ =====
260
261 void function1() {
262     Serial.println("Ejecutando Función 1");
263
264     // Adelante
265     motorsForward();
266     ledadelante();
267     delay(2000);
268
269     // Girar a la izquierda
270     motorsLeft();
271     ledizquierda();
272     delay(1000);
273
274     // Atrás
275     motorsBackward();
276     ledsatras();
277     delay(2000);
278
279     // Parar + parpadeo
280     motorsStop();
281     blinkLEDs(3, 300);
282 }
283
284 // Función 2: Giros continuos tipo "baile"
285 void function2() {
286     Serial.println("Ejecutando Función 2");
287
288     for (int i=0; i<4; i++) {
289         motorsRight();
290         ledDerecha();
291         delay(800);
292         motorsLeft();
293         ledizquierda();
294         delay(800);
295     }
296     motorsStop();
```

```

297     ledsOff();
298 }
299
300 // ===== Setup =====
301 void setup() {
302     Serial.begin(115200);
303
304     // definir el tipo de señal para los pines del driver y los leds
305     pinMode(STBY, OUTPUT);
306
307     pinMode(AIN1, OUTPUT);
308     pinMode(AIN2, OUTPUT);
309     pinMode(PWMA, OUTPUT); // Aunque se usa con ledc, es buena práctica definirlo
310
311     pinMode(BIN1, OUTPUT);
312     pinMode(BIN2, OUTPUT);
313     pinMode(PWMB, OUTPUT); // Igual que PWMA
314
315     pinMode(LED_1, OUTPUT);
316     pinMode(LED_2, OUTPUT);
317     pinMode(LED_3, OUTPUT);
318     pinMode(LED_4, OUTPUT);
319
320     digitalWrite(STBY, HIGH); // habilitar driver
321
322     // Configurar PWM en ESP32
323     ledcAttach(PWMA, pwmFree, pwmResolution);
324     ledcAttach(PWMB, pwmFree, pwmResolution);
325
326     motorsStop();
327
328     // Conexión WiFi (a tu router)
329     WiFi.begin(ssid, password);
330
331     Serial.print("Conectando a WiFi");
332     while (WiFi.status() != WL_CONNECTED) {
333         delay(400);
334         Serial.print(".");
335     }
336     Serial.println("\nConectado. IP: ");
337     Serial.println(WiFi.localIP());
338
339     // Rutas HTTP
340     server.on("/", [](){ server.send(200, "text/html", html); });
341
342     server.on("/forward", [](){ motorsForward(); server.send(200,"text/plain","Forward"); });
343     server.on("/backward", [](){ motorsBackward(); server.send(200,"text/plain","Backward"); });
344     server.on("/left", [](){ motorsLeft(); server.send(200,"text/plain","Left"); });
345     server.on("/right", [](){ motorsRight(); server.send(200,"text/plain","Right"); });
346     server.on("/stop", [](){ motorsStop(); server.send(200,"text/plain","Stop"); });
347

```

```
348 // Nuevas rutas para las funciones 1 y 2
349 server.on("/function1", [](){ function1(); server.send(200,"text/plain","Function 1 executed"); });
350 server.on("/function2", [](){ function2(); server.send(200,"text/plain","Function 2 executed"); });
351
352 // Cambiar velocidad desde el slider
353 server.on("/speed", [](){
354 if (server.hasArg("val")) {
355     motorSpeed = constrain(server.arg("val").toInt(), 0, 255);
356 }
357 server.send(200, "text/plain", String(motorSpeed));
358 });
359
360 server.begin();
361 Serial.println("Servidor HTTP listo.");
362 }
363
364 // ===== Loop =====
365 void loop() {
366     server.handleClient();
367 }
```